ATARS v2.0

System Documentation

Automated Time-Series Analysis and Reporting System

Version 2.0.0 · Release Date: 2026 · Python 3.10+

Author: Priyanshu · B.Tech Computer Science

MIT License — Open Source. Copyright (c) 2026 Priyanshu.

4,449 lines · 44 functions · 13 departments · 16 formal operators · 28 equations · 15 charts · 18-section report

Document Information

Table 1: Document version history

Version	Date	Author	Change Summary
2.0.0	2026-02-26	Priyanshu	Initial release of v2.0: ML enhancements, RGV, report writing fixes
1.5.0	2025-12	Priyanshu	Security hardening, SHA-256 audit, Task Scheduler automation
1.0.0	2025-10	Priyanshu	Initial framework: 16 operators, 9-step pipeline, Word report

Document scope: Complete technical reference for installing, configuring, running, and extending ATARS v2.0

Target audience: Researchers, data engineers, and developers working with urban air quality monitoring data

Prerequisites: Python 3.10+, pandas, numpy, scipy, scikit-learn, matplotlib, python-docx, openpyxl

Source file: atars_v2.py (4,449 lines, single-file deployment)

Contents

1. System Overview 4
2. Installation and Environment Setup 5
3. Quick Start 6
4. Configuration Reference 7
5. System Architecture 9
6. Formal Mathematical Framework 11
7. Pipeline Step-by-Step Reference 13
8. Function Reference — All 44 Functions 16
9. Machine Learning Modules 22
10. Runtime Grounding Verifier (RGV) 24
11. Output Package Reference 26
12. Report Structure — All 18 Sections 28
13. Chart Reference — All 15 Charts 30
14. Security Architecture 32
15. Windows Task Scheduler Automation 33
16. Troubleshooting Guide 34
17. Error Reference 36
18. Extending ATARS v2.0 37
19. Changelog and Known Issues 39
20. Glossary 40

1. System Overview

ATARS v2.0 is a single-file Python system (atars_v2.py) that transforms a raw urban air quality CSV file into a complete analysis package: a 18-section Word report, 15 publication-ready charts, a JSON audit log, and a Windows Task Scheduler batch file — fully automatically, with no manual steps.

The system is built around a formal mathematical framework of 16 equation-numbered operators. These operators produce a JSON data contract J which serves as the sole input to an optional LLM narrative generator. Every LLM-generated sentence is then verified against J by the Runtime Grounding Verifier (RGV), which produces a scalar G_rate metric logged in the audit trail.

1.1 Core Design Principles

- **Formal reproducibility —** Every result is a deterministic function of the input CSV and configuration. SHA-256 audit hashes verify this.
- **Security isolation —** The LLM receives aggregated statistics only — never raw sensor data. Zero external network calls when --no-llm is used.
- **Additive ML —** Machine learning modules extend but never replace the formal statistical layer. ML results are excluded from J.
- **Single-file deployment —** The entire system is one Python file. No database, no API server, no cloud dependency.
- **Graceful degradation —** Every optional step (LLM, ML, specific chart) has a fallback. The pipeline never crashes on partial data.

1.2 What ATARS v2.0 Produces

Table 2: ATARS v2.0 output summary

Output	Format	Size (typical)	Contents
Analysis Report	Word .docx	1.5–3 MB	18 sections, 15 embedded charts, narrative, RGV results
15 Charts	PNG files	300–800 KB each	Time-series, heatmaps, box plots, anomaly dashboards
JSON Contract	audit JSON	20–50 KB	SHA-256 hashes, G_rate, operator parameters, metadata
Batch File	Windows .bat	2 KB	Task Scheduler automation for scheduled re-runs

1.3 Supported Variables

Table 3: Monitored variables with physical bounds and WHO AQG thresholds

Variable	Unit	Physical Min	Physical Max	WHO AQG 2021	Column Aliases
PM10	µg/m³	0	1000	45	pm10, PM10, pm_10, particulate_10
PM2.5	µg/m³	0	500	15	pm25, PM2.5, pm_25, fine_pm
NO_2	µg/m³	0	500	10	no2, NO2, nitrogen_dioxide
SO_2	µg/m³	0	500	40	so2, SO2, sulphur_dioxide
CO	mg/m³	0	50	4	co, CO, carbon_monoxide
Ozone	µg/m³	0	300	60	o3, ozone, O3, tropospheric_ozone
Benzene	µg/m³	0	100	1.7	benzene, C6H6, btex_benzene
Humidity	%	0	100	N/A	humidity, rh, relative_humidity

2. Installation and Environment Setup

2.1 System Requirements

Table 4: System requirements

Requirement	Minimum	Recommended	Notes
Python	3.10	3.11+	f-strings, match-case, type hints used
RAM	4 GB	8 GB+	For large datasets >500K rows
Storage	500 MB	2 GB+	For chart PNGs and report output
OS	Windows 10 / Ubuntu 20.04	Windows 11 / Ubuntu 22.04	Task Scheduler .bat requires Windows
Ollama (optional)	0.1.0	Latest	Required only for LLM narrative generation

2.2 Python Dependencies

Install all required packages with a single pip command:

```
pip install pandas numpy scipy scikit-learn matplotlib seaborn
            python-docx openpyxl requests
```

Full dependency list with version constraints:

Table 5: Python package dependencies

Package	Min Version	Purpose	Import Name
pandas	1.5	Data loading, cleaning, resampling	import pandas as pd
numpy	1.23	Numerical computation, arrays	import numpy as np
scipy	1.9	Stats: ACF, PACF, OLS, CI	from scipy import stats
scikit-learn	1.1	Isolation Forest, StandardScaler	from sklearn.ensemble import ...
matplotlib	3.6	All 15 chart figures	import matplotlib.pyplot as plt
seaborn	0.12	Correlation heatmap styling	import seaborn as sns
python-docx	0.8	Word .docx report generation	from docx import Document
openpyxl	3.0	Excel output (optional)	import openpyxl
requests	2.28	Ollama LLM API calls (optional)	import requests

2.3 Ollama Setup (Optional — LLM Narrative)

Ollama is required only if AI narrative generation is enabled (use_llm: true in config). If you use --no-llm, Ollama is not needed and the system runs with zero network calls.

```
# 1. Download and install Ollama from https://ollama.com
# 2. Pull the recommended model:
ollama pull llama3.2

# 3. Verify Ollama is running:
curl http://localhost:11434/api/tags

# 4. Test ATARS with LLM enabled:
python atars_v2.py --input data.csv --city "Delhi" --use-llm
```

3. Quick Start

3.1 Minimal Run (No LLM)

The fastest way to run ATARS is with a CSV file and the --no-llm flag:

```
python atars_v2.py --input your_data.csv --city "Mumbai" --no-llm
```

This produces the complete output package (report, 15 charts, audit log) in 2–5 minutes.

3.2 CSV Format Requirements

ATARS accepts any CSV with a datetime column and at least one pollutant column. Column names are normalised automatically — you do not need to rename your columns.

```
# Minimum valid CSV:
datetime,PM10
2024-01-01 00:00,45.2
2024-01-01 01:00,48.7

# Full example CSV:
datetime,PM10,NO2,SO2,CO,Ozone,Benzene,Humidity
2024-01-01 00:00,45.2,32.1,8.5,1.2,41.0,3.2,65.0
```

3.3 Command-Line Arguments

Table 6: All command-line arguments

Argument	Type	Default	Description
--input	str	(required)	Path to input CSV file
--city	str	"Unknown"	City/station name for report title and audit
--output-dir	str	./output	Directory for all output files
--no-llm	flag	disabled	Skip LLM narrative, use statistical placeholders
--use-llm	flag	enabled	Enable LLM narrative (requires Ollama)
--llm-model	str	llama3.2	Ollama model name to use
--z-threshold	float	3.0	Z-score anomaly detection threshold ζ_thresh
--window	int	30	Rolling baseline window W (days)
--forecast-days	int	14	Holt-Winters forecast horizon (days)
--no-ml	flag	disabled	Disable both Isolation Forest and Holt-Winters
--config	str	None	Path to JSON config file (overrides all CLI args)
--version	flag	—	Print ATARS version and exit

3.4 Example Runs

```
# Full run with LLM, custom thresholds:
python atars_v2.py --input delhi_2024.csv --city "Delhi" --use-llm --z-
threshold 2.5

# Production run, no LLM, custom output directory:
python atars_v2.py --input data.csv --city "Mumbai" --no-llm --output-dir
./reports/jan2024

# Using a JSON config file:
python atars_v2.py --config atars_config.json
```

4. Configuration Reference

ATARS v2.0 is configured through the DEFAULT_CONFIG dictionary (lines 45–120 in atars_v2.py) or via an external JSON config file passed with --config. All parameters below can be set in either location.

4.1 Data Parameters

Table 7: Data configuration parameters

Parameter	Type	Default	Description
city	str	"Unknown"	Station or city name used in report title
input_file	str	None	Absolute or relative path to input CSV
output_dir	str	"./output"	Directory where all outputs are written
datetime_col	str	"datetime"	Name of the datetime column (auto-detected if absent)
min_valid_fraction	float	0.70	Minimum Q(D_v) for a variable to be included
date_start	str	None	Analysis start date YYYY-MM-DD (None = CSV start)
date_end	str	None	Analysis end date YYYY-MM-DD (None = CSV end)

4.2 Statistical Operator Parameters

Table 8: Statistical analysis parameters (formal operators)

Parameter	Type	Default	Equation	Description
baseline_window_W	int	30	Eq. 6–7	Rolling baseline window width W (days)
z_threshold	float	3.0	Eq. 8	Z-score anomaly threshold ζ_thresh
ci_level	float	0.95	Eq. 10	Confidence interval level (0.95 = 95%)
acf_lags	int	30	Eq. 11	Maximum ACF/PACF lag to compute and plot
ols_target	str	"PM10"	Eq. 14	Target variable for OLS regression
min_valid	float	0.70	Eq. 2	Minimum Q(D_v) to include variable in analysis

4.3 Machine Learning Parameters

Table 9: ML module parameters

Parameter	Type	Default	Module	Description
use_isolation_forest	bool	True	IF	Enable Isolation Forest anomaly detection

Parameter	Type	Default	Module	Description
if_n_estimators	int	150	IF	Number of isolation trees in ensemble
if_contamination	float	0.05	IF	Expected fraction of anomalies (5%)
if_random_state	int	42	IF	Random seed for reproducibility (fixed)
use_hw_forecast	bool	True	HW	Enable Holt-Winters PM10 forecast
hw_alpha	float	0.30	HW	Level smoothing parameter $\alpha \in (0,1)$
hw_beta	float	0.10	HW	Trend smoothing parameter $\beta \in (0,1)$
hw_gamma	float	0.20	HW	Seasonal smoothing parameter $\gamma \in (0,1)$
hw_seasonal_periods	int	7	HW	Seasonality period m (7 = weekly)
hw_forecast_horizon	int	14	HW	Forecast horizon in days
hw_pi_level	float	0.80	HW	Prediction interval confidence level (80%)

4.4 LLM Parameters

Table 10: LLM narrative generation parameters

Parameter	Type	Default	Description
use_llm	bool	True	Enable LLM narrative generation
llm_model	str	"llama3.2"	Ollama model name
ollama_url	str	"http://localhost:11434"	Ollama server URL
llm_timeout	int	120	Per-section timeout in seconds
llm_retry	int	2	Number of retries on timeout
llm_retry_delay	int	5	Seconds to wait between retries
llm_temperature	int	0	Generation temperature (0 = deterministic)
llm_max_tokens	int	512	Maximum tokens per narrative section

4.5 JSON Config File Example

Save the following as atars_config.json and pass with --config atars_config.json:

```
{
  "city": "Delhi",
  "input_file": "delhi_2024.csv",
  "output_dir": "./output/delhi_2024",
  "use_llm": false,
```

```
  "z_threshold": 3.0,
  "baseline_window_W": 30,
  "use_isolation_forest": true,
  "if_n_estimators": 150,
  "if_contamination": 0.05,
  "if_random_state": 42,
  "use_hw_forecast": true,
  "hw_alpha": 0.30,
  "hw_beta": 0.10,
  "hw_gamma": 0.20,
  "hw_forecast_horizon": 14
}
```

5. System Architecture

5.1 Module Structure

ATARS v2.0 is organised as a single Python file with 13 named departments. Each department is separated by a prominent ASCII banner comment in the source code.

Table 11: ATARS v2.0 source code department map

Dept.	Name	Lines (approx.)	Key Functions
01	Configuration & Constants	45–180	DEFAULT_CONFIG, WHO thresholds, colour palettes
02	Data Loading & Normalisation	181–350	load_data(), assign_quality_flags(), normalise_columns()
03	Daily Statistics	351–550	build_daily_stats(), compute_baseline(), compute_zscore()
04	Temporal & Correlation	551–720	run_temporal_analysis(), compute_acf_pacf(), compute_ols()
05	JSON Contract Builder	721–850	build_json_contract(), flatten_to_j()
06	LLM Narrative Engine	851–1050	try_llm_narrative(), _placeholder_text(), _clean_llm_response()
07	Runtime Grounding Verifier	1051–1400	RuntimeGroundingVerifier class, verify_narrative()
08	Chart Generation	1401–2100	chart_timeseries() … chart_forecast() — 15 functions
09	Chart Insights Engine	2101–2250	generate_chart_insights()
10	ML: Isolation Forest	2251–2400	run_isolation_forest(), chart_ml_anomaly()
11	ML: Holt-Winters Forecast	2401–2550	run_hw_forecast(), chart_forecast()
12	Report Builder	2551–3900	build_report() — 18-section Word document
13	Audit & Automation	3901–4100	write_audit_log(), generate_bat_file(), run_pipeline()

5.2 Data Flow

The following diagram shows the complete data flow through the ATARS v2.0 pipeline:

> **INPUT LAYER**
> *CSV File → load_data() → normalise_columns() → assign_quality_flags()*
> *Output: validated DataFrame with q_{t,v} flags*

STATISTICAL LAYER (Formal Operators Eqs. 1–14)
build_daily_stats() → daily_df [x̄_d, σ_d, ζ_d, δ_d, CI, Q(D)]
run_temporal_analysis() → ACF, PACF, Pearson r, OLS β̂
build_json_contract() → JSON contract J [ML results EXCLUDED]

ML LAYER (Additive, not in J)
run_isolation_forest() → IF anomaly flags + scores per day
run_hw_forecast() → 14-day PM10 forecast + 80% PI

LLM LAYER (receives J only)
try_llm_narrative() → 6-section narrative dict
_clean_llm_response() → strip preamble hallucinations
RuntimeGroundingVerifier → G_rate + evidence table + verdicts

OUTPUT LAYER
generate_all_charts() → 15 PNG files
build_report() → ATARS_Report.docx (18 sections)
write_audit_log() → atars_audit_{hash}.json
generate_bat_file() → run_atars.bat

5.3 The JSON Contract J

J is the formally defined boundary between the statistical layer and the LLM. It is constructed by build_json_contract() and contains only aggregated statistics. J is never modified after creation. The canonical JSON serialisation (sort_keys=True, ensure_ascii=True) produces the SHA-256 hash_J.

J top-level structure:

```
{
  "city": "Delhi",
  "date_start": "2024-01-01",
  "date_end": "2024-12-31",
  "total_days": 366,
  "data_quality": { "overall_Q": 0.924, "confidence_flag": "HIGH", ... },
  "variables": {
    "PM10": { "mean": 87.3, "std": 42.1, "max": 312.0, "min": 8.2,
              "threshold": 45.0, "exceed_days": 142, "anomaly_days": 12,
              "mean_zscore": 0.12, "Q": 0.942 },
    ...
  },
  "formal_operators": { "z_threshold": 3.0, "baseline_window_W": 30, ... },
  "correlations": { "PM10_NO2": 0.71, "PM10_SO2": 0.63, ... },
  "ols_result": { "R2": 0.48, "PM10_humidity": -0.31, ... },
  "top_correlations": [ ["PM10","NO2",0.71], ... ],
  "top_pollutant": "PM10",
  "exceedances": { "PM10": 142, "NO2": 38, ... },
  "ai_constraints": { "prohibit_causation": true, "prohibit_forecast": true }
}
```

6. Formal Mathematical Framework

ATARS v2.0 defines 16 formally numbered operators. Each operator has a unique equation number (Eq. 1–17), a corresponding Python function, and appears in the report with its equation label. This section provides the complete operator reference for technical users.

Table 12: Complete operator reference — all 16 operators

Eq.	Operator	Symbol	Formula (summary)	Function	Section in Report
1	Quality Flag	$q_{t,v}$	1 if $l_v \le x \le u_v$ else 0	assign_quality_flags()	Sec 01
2	Quality Score	$Q(D_v)$	N_{valid} / N_{total}	build_daily_stats()	Sec 01
3	Daily Mean	$\bar{x}_{d,v}$	$(1/N) \Sigma x_{t,v}$	build_daily_stats()	Sec 03
4	Daily Std Dev	$\sigma_{d,v}$	$\mathrm{sqrt}[\Sigma(x-\bar{x})^2 / (N-1)]$	build_daily_stats()	Sec 03
5	Valid Record Count	$N_{d,v}$	$\lvert\{t \in d : q_{t,v}=1\}\rvert$	build_daily_stats()	Sec 01
6	Rolling Baseline Mean	$\bar{x}_{W,d}$	$(1/\lvert W_d \rvert) \Sigma \bar{x}_{d'}$	compute_baseline()	Sec 03
7	Baseline Std Dev	$\sigma_{W,d}$	$\mathrm{sqrt}[\Sigma(\bar{x}_{d'}-\bar{x}_{W,d})^2 / (\lvert W \rvert -1)]$	compute_baseline()	Sec 03
8	Z-Score	$\zeta_{d,v}$	$(\bar{x}_d - \bar{x}_W) / \sigma_W$	compute_zscore()	Sec 04
9	Delta Deviation	$\delta_{d,v}$	$[(\bar{x}_d - \bar{x}_W) / \bar{x}_W] \times 100\%$	compute_delta()	Sec 04
10	Confidence Interval	$CI_{d,v}$	$\bar{x}_d \pm 1.96\, \sigma_d/\sqrt{N}$	compute_ci()	Sec 04
11	ACF	$\hat{\rho}(k)$	$Cov(x_t,x_{t-k}) / Var(x)$	compute_acf_pacf()	Sec 05
12	PACF	$\hat{\phi}_{kk}$	Yule-Walker: $\Phi_k = R_k^{-1} r_k$	compute_acf_pacf()	Sec 05
13	Pearson Correlation	$r_{u,v}$	$\Sigma(x-\bar{x})(y-\bar{y}) / [(n-1)\sigma_x\sigma_y]$	compute_correlations()	Sec 06
14	OLS Regression	$\hat{\beta}$	$(X^TX)^{-1} X^Ty$	compute_ols()	Sec 11
15	Grounding Property	$G(s) \subseteq J$	$\forall c \in C(s): \exists j \in J: \lvert c-j \rvert / \lvert j \rvert \le \varepsilon$	verify_section()	Sec 13
16	G_rate Metric	G_{rate}	$\lvert\{s:G(s) \subseteq J\}\rvert / \lvert N_{num} \rvert$	verify_narrative()	Sec 13
17	Reproducibility	$hash_J$	SHA-256(canonical_JSON(J))	write_audit_log()	Sec 18

7. Pipeline Step-by-Step Reference

The main entry point run_pipeline() orchestrates all 9 steps. Each step is logged to the console with a ✓ or ⚠ indicator. Failures in optional steps (ML, LLM) do not abort the pipeline.

Step 1: Load and Normalise

Function: load_data(config)

Input: CSV file at config["input_file"]

Output: Validated pandas DataFrame with canonical column names

Reads the CSV with pandas read_csv(), auto-detects the datetime column, and applies normalise_columns() to map all synonym column names to canonical names. Timezone-aware parsing handles UTC offsets. Invalid datetime rows are dropped with a warning.

```
Console output:
  [Step 1] Loading data from delhi_2024.csv ...
  ✓  Loaded 61,320 records | 7 variables | 2024-01-01 to 2024-12-31
  ✓  Columns mapped: PM10, NO2, SO2, CO, Ozone, Benzene, Humidity
```

Step 2: Quality Flag Assignment

Function: assign_quality_flags(df, config)

Operator: Eq. 1: q_{t,v} = 1 if l_v ≤ x_{t,v} ≤ u_v else 0

Output: DataFrame with q_{var}_flag boolean columns added

Applies physical bounds per variable. Records outside [l_v, u_v] receive q=0 and are excluded from all downstream statistical operators. The number of excluded records per variable is logged.

Step 3a: Core Statistical Computation

Function: build_daily_stats(df, config)

Operators: Eqs. 2–10: Q(D_v), x̄_d, σ_d, N_d, x̄_W, σ_W, ζ_d, δ_d, CI

Output: daily_df: one row per (day, variable) with all computed statistics

This is the computational core of ATARS. For each (day, variable) pair, it computes: daily mean, standard deviation, valid record count, data quality Q(D_v), 30-day rolling baseline (mean and std), z-score anomaly flag, delta percentage deviation, and 95% confidence interval bounds. Days with zero valid records are filled with NaN and excluded from baseline computation.

Step 3c: Isolation Forest (Optional ML)

Function: run_isolation_forest(daily_df, config)

Operator: Eq. 18: s(x,n) = 2^{-E[h(x)]/c(n)}

Output: DataFrame column if_anomaly (bool) + if_score (float)

Pivots daily_df to a (n_days × 5) feature matrix, applies StandardScaler, fits IsolationForest with n_estimators=150, contamination=0.05, random_state=42. Returns per-day anomaly flags and raw IF scores. Produces the BOTH / IF-only / z-score-only / NORMAL categorisation.

Disabled with --no-ml or use_isolation_forest: false in config.

Step 3d: Holt-Winters Forecast (Optional ML)

Function: run_hw_forecast(daily_df, config)

Operators: Eqs. 19–23: Level, Trend, Seasonal updates + forecast + 80% PI

Output: Dictionary with forecast array, lower/upper PI bounds, RMSE, MAE

Fits the HW additive model to the daily PM10 series using the configured α, β, γ parameters. Generates a 14-day ahead forecast with 80% prediction interval using σ_{resid} × √h widening. Returns in-sample RMSE and MAE for the audit log. Excluded from J.

Step 4: Temporal and Multivariate Analysis

Function: run_temporal_analysis(daily_df, config)

Operators: Eqs. 11–14: ACF, PACF, Pearson r, OLS $\hat{\beta}$

Output: acf_dict, pacf_dict, corr_matrix DataFrame, ols_result dict

Step 5: JSON Contract Construction

Function: build_json_contract(daily_df, corr_matrix, ols_result, config)

Operator: Eq. 17 (applied after): SHA-256(canonical_JSON(J))

Output: J dict — the formal data boundary

Assembles J from aggregated statistics only. Explicitly excludes ML results. Validates that all required fields are present before returning.

Step 6: Chart Generation

Function: generate_all_charts(daily_df, J, corr_matrix, ols_result, hw_result, if_result, insights, config)

Output: 15 PNG files saved to output_dir

Each chart function receives only the data it needs. See Section 13 for the full chart reference.

Step 7: LLM Narrative Generation

Function: try_llm_narrative(J, config)

Output: narrative dict with keys: A_summary, B_pollutants, C_correlations, D_anomalies, E_quality, F_limitations

Attempts to connect to Ollama at config["ollama_url"]. On connection failure, timeout, or --no-llm flag, falls back to _placeholder_narrative(J) which generates statistical placeholders from J values with zero hallucination risk. LLM responses are cleaned by _clean_llm_response() before storage.

Step 8: Report Generation

Function: build_report(J, narrative, verification, daily_df, corr_matrix, ols_result, hw_result, if_result, chart_paths, insights, config)

Output: ATARS_Report_{City}_{Date}.docx — 18-section Word document

The largest function in the codebase (~1,350 lines). See Section 12 for the full report structure.

Step 9: RGV Verification and Audit

Function: RuntimeGroundingVerifier(J).verify_narrative(narrative)

Operator: Eqs. 15–16, 24–26: G(s)⊆J, G_rate, grounded(c,j)

Output: verification dict with G_rate, evidence table, verdict per section

Runs after narrative generation. Flattens J, extracts all numerical claims from every narrative sentence, matches claims against J within type-adaptive tolerance ε, assigns sentence-level verdicts, and computes the overall G_rate. Results are embedded in the report Section 13 and saved to the audit log.

8. Function Reference — All 44 Functions

This section documents every function in atars_v2.py. Functions are listed by department. Signatures follow Python 3.10 type annotation conventions.

8.1 Department 01 — Configuration & Constants

get_config(args) → dict

Merges DEFAULT_CONFIG with CLI arguments and any JSON config file. Returns the resolved config dict. CLI arguments override JSON config; JSON config overrides DEFAULT_CONFIG.

validate_config(config) → None

Validates all config values (type checks, range checks, path existence). Raises ValueError with a descriptive message on any violation.

load_who_thresholds() → dict

Returns a dict mapping variable names to WHO AQG 2021 annual guideline values (µg/m³). Used by quality reporting and chart annotations.

8.2 Department 02 — Data Loading & Normalisation

load_data(config) → pd.DataFrame

Reads the CSV, detects datetime column, applies timezone-aware parsing, and calls normalise_columns(). Returns a validated DataFrame. Raises FileNotFoundError, ValueError on parse failure.

normalise_columns(df) → pd.DataFrame

Maps all column names to canonical names using a regex-based synonym dictionary. Case-insensitive. Logs all mappings to console. Returns the renamed DataFrame.

assign_quality_flags(df, config) → pd.DataFrame

Applies Eq. 1 physical bounds per variable. Adds boolean q_{var}_flag column for each variable. Records outside bounds receive False. Returns the augmented DataFrame.

infer_sampling_frequency(df) → str

Infers hourly, 30-min, or daily sampling frequency from the datetime index median interval. Returns a pandas freq string ("1H", "30T", "1D").

8.3 Department 03 — Daily Statistics

build_daily_stats(df, config) → pd.DataFrame

Core computation function. Applies Eqs. 3–10 for each (day, variable) pair. Returns daily_df with columns: mean, std, n_valid, Q, baseline_mean, baseline_std, zscore, delta, ci_lower, ci_upper, anomaly_flag.

compute_baseline(daily_df, W) → pd.DataFrame

Applies Eqs. 6–7 rolling baseline. Uses pandas rolling(W, center=True, min_periods=W//3) to handle edge effects. Returns daily_df with baseline_mean and baseline_std columns.

compute_zscore(daily_df) → pd.DataFrame

Applies Eq. 8. Returns daily_df with zscore and anomaly_flag (|ζ|>ζ_thresh) columns.

compute_delta(daily_df) → pd.DataFrame

Applies Eq. 9. Returns daily_df with delta_pct column. Handles baseline_mean=0 with NaN to prevent division by zero.

compute_ci(daily_df, ci_level) → pd.DataFrame

Applies Eq. 10 using scipy.stats.t.ppf for the critical value (uses t-distribution for N<30). Returns daily_df with ci_lower and ci_upper columns.

compute_quality_scores(daily_df, n_total_per_day) → pd.DataFrame

Applies Eq. 2 per variable per day, then computes overall Q(D) = min_v Q(D_v). Returns daily_df with Q_var and Q_overall columns.

8.4 Department 04 — Temporal and Correlation Analysis

run_temporal_analysis(daily_df, config) → tuple

Orchestrator for Eqs. 11–14. Returns (acf_dict, pacf_dict, corr_matrix, ols_result).

compute_acf_pacf(series, n_lags) → tuple[np.ndarray, np.ndarray]

Applies Eqs. 11–12 using statsmodels acf/pacf with alpha=0.05 significance bands. Returns (acf_values, pacf_values, confidence_bands).

compute_correlations(daily_df) → pd.DataFrame

Applies Eq. 13. Computes the full Pearson correlation matrix across all variables using daily means. Returns a square DataFrame indexed and columned by variable name.

compute_ols(daily_df, target, predictors, config) → dict

Applies Eq. 14. Standardises predictors with z-score normalisation, fits OLS via numpy.linalg.lstsq, computes R^2, and returns {R2, intercept, coefficients, p_values, residuals}.

8.5 Department 05 — JSON Contract

build_json_contract(daily_df, corr_matrix, ols_result, config) → dict

Assembles J from all aggregated statistics. Explicitly excludes ML results (if_result, hw_result). Validates completeness. Returns the formal data contract dict.

flatten_j(j, path="") → dict[str, float]

Recursively flattens J to {dotted.path: float_value} for the RGV lookup table. Used internally by RuntimeGroundingVerifier.__init__().

8.6 Department 06 — LLM Narrative Engine

try_llm_narrative(J, config) → dict

Attempts Ollama connection, generates 6 narrative sections sequentially, applies _clean_llm_response() to each. Falls back to _placeholder_narrative(J) on any error.

_placeholder_narrative(J) → dict

Generates all 6 narrative sections as statistical placeholder strings from J values. Zero LLM dependency. All numerical values are sourced directly from J. Used when --no-llm is set.

_placeholder_text(key, J) → str

Returns a single placeholder paragraph for section key (A_summary … F_limitations). All values interpolated from J with f-strings.

_clean_llm_response(text) → str

Strips 14 common LLM preamble patterns from the response text. Applies a 25% minimum-length retention guard. Recapitalises the first letter if needed. Returns the cleaned string.

8.7 Department 07 — Runtime Grounding Verifier

class RuntimeGroundingVerifier

Primary novel contribution class. Implements G(s)⊆J enforcement and G_rate computation (Eqs. 15–16, 24–26).

__init__(self, J)

Builds the numeric lookup table L = {path: float} and string_lookup via _build_lookup(). Records |L| for the audit log.

verify_section(self, section_key, text) → dict

Verifies one narrative section. Returns {section_key, sentences, total_sentences, numerical_sentences, grounded_sentences, ungrounded_sentences, G_rate, passes, verdict}.

verify_narrative(self, narrative) → dict

Verifies the full narrative dict. Returns the complete verification result with overall G_rate, evidence table, and ungrounded_list (up to 500 chars per sentence with ellipsis).

extract_claims(self, sentence) → list[dict]

Extracts numerical claims from a sentence using regex. Returns list of {raw, value, type, context}. Skips citation-style numbers <30.

check_claim(self, claim) → dict

Matches a single claim against L using type-adaptive tolerance ε. Returns {matched, best_match_path, best_match_value, ratio}.

8.8 Departments 08–09 — Chart Generation

generate_all_charts(…) → dict[str, str]

Orchestrator. Calls all 15 chart functions, handles per-chart errors gracefully, returns {chart_key: file_path} dict.

generate_chart_insights(daily_df, J, corr_matrix, hw_result, if_result) → dict

Generates all 15 chart insight strings deterministically from statistics — no LLM. All numerical values are directly from daily_df or J. Returns {chart_key: insight_text}.

Individual chart functions (chart_01 through chart_15) are documented in Section 13.

8.9 Departments 10–11 — ML Modules

run_isolation_forest(daily_df, config) → pd.DataFrame

See Section 9.1 for full documentation.

run_hw_forecast(daily_df, config) → dict

See Section 9.2 for full documentation.

8.10 Department 12 — Report Builder

build_report(…) → str

Builds the 18-section Word document. Uses python-docx internally. Returns the output file path. Each section is built by a helper sub-function (section_01() … section_18()). See Section 12.

8.11 Department 13 — Audit and Automation

write_audit_log(J, verification, hw_result, if_result, config) → str

Computes hash_J (Eq. 27) and hash_report (Eq. 28), assembles the audit record, and writes atars_audit_{hash_J[:12]}.json. Returns the file path.

generate_bat_file(config) → str

Generates a Windows .bat file that re-runs ATARS with the same config. Includes Task Scheduler XML registration commands. Returns the file path.

run_pipeline(config) → None

Main entry point. Executes all 9 steps in order with console progress logging. Steps 3c, 3d, and 7 are wrapped in try/except for graceful degradation.

9. Machine Learning Modules

9.1 Isolation Forest — Multivariate Anomaly Detection

The Isolation Forest module detects multivariate anomalies that univariate z-score analysis misses. It operates on a (n_days × 5) feature matrix of [PM10, NO_2, SO_2, CO, Ozone] daily means.

Algorithm (Eq. 18)

Anomaly score: s(x,n) = 2^{-E[h(x)]/c(n)} where E[h(x)] = mean path length across 150 trees.

Parameters

Table 13: Isolation Forest parameters

Parameter	Value	Rationale
n_estimators	150	Higher than default (100) for more stable scores on annual data
contamination	0.05	5% expected anomaly rate (≈18 days/year) — conservative for pollution
random_state	42	Fixed seed for complete reproducibility — matches SHA-256 audit guarantee
StandardScaler	Yes	Z-normalise each feature so PM10 scale dominates CO scale

Output Categories

Table 14: IF vs z-score comparison categories

Category	IF Flag	ζ Flag	Interpretation	Confidence
BOTH	True	True	Both methods agree — confirmed anomaly	**HIGH**
IF-only	True	False	Multivariate pattern, no single spike	**MEDIUM**
z-score only	False	True	Univariate spike within normal combination	**MEDIUM**
NORMAL	False	False	Neither method flagged	—

Security Note

IF results are excluded from J. The LLM cannot access IF anomaly flags or scores. IF results appear only in Chart 14 and Report Section 14, both clearly labelled as supplementary ML analysis.

9.2 Holt-Winters — 14-Day PM10 Forecast

Triple exponential smoothing applied to the daily PM10 series to produce a 14-day ahead forecast with 80% prediction interval.

Update Equations

Level: $L_t = \alpha(x_t - S_{t-m}) + (1-\alpha)(L_{t-1}+T_{t-1})$ [Eq. 19]

Trend: $T_t = \beta(L_t - L_{t-1}) + (1-\beta)T_{t-1}$ [Eq. 20]

Seasonal: $S_t = \gamma(x_t - L_t) + (1-\gamma)S_{t-m}$ [Eq. 21]

Forecast: $\hat{x}_{t+h} = L_t + h \cdot T_t + S_{t-m+((h-1) \bmod m)+1}$ [Eq. 22]

80% PI: $\hat{x}_{t+h} \pm 1.28 \times \sigma_{resid} \times \sqrt{h}$ [Eq. 23]

Initialisation

L_0 = mean(first week), T_0 = (mean(second week) - mean(first week)) / m, $S_i = x_i - L_0$ for i=1…7.

Output

```
hw_result = {
  "forecast":   np.ndarray (14,),   # point forecast
  "lower_pi":   np.ndarray (14,),   # 80% PI lower bound
  "upper_pi":   np.ndarray (14,),   # 80% PI upper bound
  "rmse":       float,              # in-sample RMSE µg/m³
  "mae":        float,              # in-sample MAE µg/m³
  "residuals":  np.ndarray (n,),    # in-sample residuals
  "alpha":      0.30,
  "beta":       0.10,
  "gamma":      0.20
}
```

10. Runtime Grounding Verifier (RGV)

10.1 Overview

The RGV is ATARS v2.0's primary novel contribution. It algorithmically enforces and measures the grounding constraint G(s)⊆J (Eq. 15) for all LLM-generated narrative sentences, producing a scalar G_rate metric (Eq. 16) included in the audit log.

10.2 Tolerance Values

Table 15: Type-adaptive tolerance ε by claim type

Claim Type	Tolerance	Example Values	Rationale
Concentration (float)	ε = 5% relative	PM10 µg/m³, NO_2 µg/m³	Float rounding and unit conversions
Count (integer)	ε_abs = 2	exceed_days, anomaly_days	Off-by-one and rounding differences
Score / ratio (float)	ε_abs = 0.10	Q(D), R^2, correlation r	Small floating-point precision
Year / date	Exact match	2024, 366	Dates should be exact

10.3 Verdict Definitions

Table 16: Sentence-level RGV verdict definitions

Verdict	Condition	Counts Towards G_rate	Action
GROUNDED	All numerical claims match J within ε	YES (numerator)	Include in report normally
PARTIALLY_GROUNDED	Some claims match, some do not	YES (numerator)	Include; note in evidence
UNGROUNDED	No claims match any value in J	NO	Flag in report; list in ungrounded section
NON_NUMERICAL	Sentence has no numerical claims	N/A (excluded)	Include without verification
UNCERTAIN	Single ambiguous claim	NO	Include; mark uncertain in evidence

10.4 G_rate Thresholds

Table 17: G_rate interpretation

G_rate Range	Verdict	Meaning	Recommended Action
0.90 – 1.00	**PASS ✓**	Excellent grounding	Publish report as-is
0.75 – 0.89	**PASS ✓**	Good grounding	Review ungrounded sentences only
0.50 – 0.74	**REVIEW ⚠**	Moderate grounding failure	Expert review of all ungrounded sentences
0.00 – 0.49	**REVIEW ⚠**	Severe grounding failure	Do not use LLM narrative; use --no-llm

11. Output Package Reference

11.1 Output Directory Structure

```
output/delhi_2024/
├── ATARS_Report_Delhi_2024-12-31.docx     ← Main report (18 sections)
├── chart01_timeseries.png
├── chart02_correlation.png
├── chart03_trend.png
├── chart04_anomalies.png
├── chart05_acf_pacf.png
├── chart06_quality.png
├── chart07_seasonal.png
├── chart08_diurnal.png
├── chart09_day_of_week.png
├── chart10_voc.png
├── chart11_rain_scatter.png
├── chart12_calendar.png
├── chart13_grounding.png
├── chart14_ml_anomaly.png
├── chart15_forecast.png
├── atars_audit_1a2b3c4d5e6f.json          ← Audit log (hash in filename)
└── run_atars.bat                           ← Task Scheduler batch file
```

11.2 Audit Log Structure

The audit log atars_audit_{hash}.json contains:


```
{
  "schema": "ATARS-AUDIT-2.0",
  "timestamp": "2024-12-31T23:59:59",
  "atars_version": "2.0.0",
  "python_version": "3.11.0",
  "city": "Delhi",
  "input_file": "delhi_2024.csv",
  "hash_J": "sha256:1a2b3c...",
  "hash_report": "sha256:4d5e6f...",
  "total_records": 61320,
  "variables": ["PM10","NO2","SO2","CO","Ozone","Benzene","Humidity"],
  "overall_Q": 0.924,
  "operator_params": { "z_threshold": 3.0, "W": 30 },
  "ml_if": { "n_estimators":150, "contamination":0.05, "anomaly_days":18 },
  "ml_hw": { "alpha":0.30, "beta":0.10, "gamma":0.20, "rmse":12.4, "mae":9.8
},
  "grounding": {
    "overall_G_rate": 0.891,
    "verdict": "PASS",
    "total_numerical": 24,
    "total_grounded": 21,
    "sections": { "A_summary": {"G_rate":1.0}, ... }
  },
  "library_versions": { "pandas":"2.0.0", "sklearn":"1.3.0", ... }
}
```

12. Report Structure — All 18 Sections

Every ATARS v2.0 report contains exactly 18 sections in the following order:

Table 18: Report section reference

#	Section Title	Content	Data Source	Has Chart?
01	Data Quality Assessment	Q(D_v) per variable, confidence table, physical bounds violated	J quality block	Chart 06
02	Executive Summary	Key findings: worst pollutant, total exceed days, G_rate, quality	J aggregated	No
03	Pollutant Time-Series	Annual daily means, WHO guideline overlay, annual mean summary table	daily_df + J	Charts 01, 03
04	Anomaly Detection Results	Z-score anomaly table, ζ_d distribution, CI band explanation	daily_df zscore	Chart 04
05	Temporal Dependency	ACF/PACF with interpretation, seasonality findings	acf_dict, pacf_dict	Chart 05
06	Correlation Analysis	Pearson r matrix, top correlations, causation disclaimer	corr_matrix	Chart 02
07	Seasonal Patterns	Monthly box plots, worst/best months, seasonal trend narrative	daily_df monthly	Chart 07
08	Diurnal Profile	Hourly mean by variable, peak hours identified	hourly_df diurnal	Chart 08
09	Day-of-Week Patterns	Mean ± σ by day, weekend vs weekday comparison	daily_df DOW	Chart 09
10	VOC / BTEX Analysis	Benzene, toluene daily + monthly, carcinogen threshold comparison	daily_df voc	Chart 10
11	Rainfall & OLS Regression	PM10 vs rainfall scatter, OLS $\hat{\beta}$, R^2, meteorological drivers	ols_result	Chart 11
12	Exceedance Calendar	Annual PM10 heatmap calendar with exceed day count	daily_df exceed	Chart 12
13	Grounding Verification	RGV G_rate dashboard, section verdicts, evidence table, ungrounded	verification dict	Chart 13
14	ML Anomaly Analysis	IF vs z-score: BOTH/IF-only/z-only/NORMAL, agreement matrix	if_result	Chart 14

#	Section Title	Content	Data Source	Has Chart?
15	Forecast	HW 14-day PM10 forecast, 80% PI, RMSE, MAE, residuals	hw_result	Chart 15
16	AI Narrative	6-section LLM narrative (A–F), grounding note, model info	narrative dict	No
17	Technical Appendix	All operator equations, config parameters, library versions	config + operator defs	No
18	Audit Trail	SHA-256 hashes, G_rate per section, run metadata	audit log	No

13. Chart Reference — All 15 Charts

Table 19: Complete chart reference

#	File	Title	Type	Key Features
01	chart01_timeseries.png	Multi-Panel Hourly Concentrations	Line	WHO guideline dashed lines, data gap markers, annual mean annotation
02	chart02_correlation.png	Pearson Correlation Heatmap	Heatmap	Full |V|×|V| matrix, colour-coded r values, significance stars
03	chart03_trend.png	Daily Trend + Baseline Ribbon	Line + Fill	30-day baseline ribbon, anomaly dots (red=above, blue=below)
04	chart04_anomalies.png	CI Band + Z-Score Dashboard	Multi-panel	Top: CI ribbon + daily means. Middle: signed z-score bar. Bottom: anomaly count
05	chart05_acf_pacf.png	ACF and PACF (30 lags)	Stem	±1.96/√n significance bands, lag-7 weekly marker, lag-1 annotation
06	chart06_quality.png	Data Quality Q(D_v) by Variable	Horizontal bar	Colour-coded HIGH/MODERATE/LOW, dotted threshold lines at 0.70 and 0.90
07	chart07_seasonal.png	Monthly Box Plots	Box + violin	Median, IQR, whiskers, outliers, WHO threshold line, worst-month label
08	chart08_diurnal.png	Hourly Mean Profile	Line + shaded	All variables normalised 0-1 overlay + absolute PM10 profile
09	chart09_dow.png	Day-of-Week Means ± σ	Bar + error	Mon–Sun bars, weekend vs weekday significance band
10	chart10_voc.png	BTEX Daily and Monthly	Multi-panel	Daily benzene time-series + monthly bar, WHO 1.7 µg/m³ line
11	chart11_rain.png	Rainfall vs PM10 Scatter + OLS	Scatter	OLS regression line, R^2 annotation, 95% CI band, Pearson r
12	chart12_calendar.png	Annual PM10 Exceedance Calendar	Heatmap	52×7 GitHub-style calendar, WHO threshold colour scale, exceed count
13	chart13_grounding.png	RGV Grounding Dashboard (Novel)	Multi-panel	Top: G_rate gauge. Middle: per-section G_rate bar. Bottom: verdict distribution

#	File	Title	Type	Key Features
14	chart14_ml_anomaly.png	IF vs Z-Score Comparison	Multi-panel	Top: agreement timeline. Middle: comparison scatter. Bottom: BOTH/IF-only etc. table
15	chart15_forecast.png	HW 14-Day PM10 Forecast	Line + PI	Historical series, 80% PI ribbon, point forecast, residuals panel

14. Security Architecture

Table 20: Security properties of ATARS v2.0

Property	Status	Details
Zero external transmission	**GUARANTEED**	Only localhost:11434 (Ollama). Disabled by --no-llm. No telemetry.
Raw data isolation	**GUARANTEED**	LLM receives only J (aggregated). Raw CSV never transmitted.
ML model non-persistence	**GUARANTEED**	IF model fit and discarded in-memory. No weights written to disk.
ML exclusion from J	**GUARANTEED**	IF flags and HW forecasts excluded from J. LLM cannot access them.
Deterministic ML output	**GUARANTEED**	IF random_state=42, HW fixed α,β,γ. Same data = same ML output.
SHA-256 tamper detection	**GUARANTEED**	hash_J in audit log. Any data or config change = different hash.
LLM preamble cleaning	**GUARANTEED**	_clean_llm_response() strips 14 preamble patterns before storage.
Air-gapped compatible	**SUPPORTED**	Use --no-llm for fully offline operation.

15. Windows Task Scheduler Automation

ATARS generates a run_atars.bat file that can be registered with Windows Task Scheduler for automated weekly or monthly runs.

15.1 Generated Batch File

```
@echo off
REM ATARS v2.0 Automated Run — Generated 2026-02-26
REM MIT License — Open Source. Copyright (c) 2026 Priyanshu.

set PYTHON=python
set SCRIPT=C:\atars\atars_v2.py
set INPUT=C:\data\delhi_latest.csv
set OUTPUT=C:\reports\latest
set CITY=Delhi

%PYTHON% %SCRIPT% --input %INPUT% --output-dir %OUTPUT% --city "%CITY%" --no-
llm

if %errorlevel% neq 0 (
    echo ATARS run FAILED at %date% %time% >> C:\logs\atars_errors.log
) else (
    echo ATARS run OK at %date% %time% >> C:\logs\atars_runs.log
)
```

15.2 Registering with Task Scheduler

```
schtasks /create /tn "ATARS Weekly Run" ^
  /tr "C:\atars\run_atars.bat" ^
  /sc WEEKLY /d MON /st 06:00 ^
  /ru SYSTEM /rl HIGHEST
```

16. Troubleshooting Guide

16.1 Common Issues

Table 21: Common issues and solutions

Issue	Symptom	Solution
No datetime column found	"ValueError: No datetime column detected"	Add a column named "datetime" or "timestamp". ATARS also accepts "date", "time", "Date", "DateTime"
All records quality-flagged	Q(D) = 0.00 for all variables	Check physical bounds in config. Your data may use different units (e.g., CO in µg/m³ not mg/m³)
Ollama not running	"Ollama not running — using placeholders"	Run: ollama serve. Or use --no-llm to skip LLM completely
Model not found	"llama3.2 not found in Ollama"	Run: ollama pull llama3.2. Or set llm_model to an installed model name
Memory error on large CSV	MemoryError during load_data()	Filter the CSV to the target year before loading. ATARS processes one year at a time
Chart PNG not saved	"chart04 skipped: insufficient data"	Need ≥30 days with valid data for baseline-dependent charts. Check quality flags
G_rate = 0.0	All sentences ungrounded	LLM is ignoring J. Try a stronger model (llama3.1:70b) or use --no-llm
Report not opening	Word shows "corrupt file" error	python-docx version mismatch. pip install python-docx==0.8.11

16.2 Diagnostic Commands

```
# Check ATARS version and environment:
python atars_v2.py --version

# Validate CSV format before running:
python -c "import pandas as pd; df=pd.read_csv('data.csv'); print(df.dtypes)"

# Check Ollama status:
curl http://localhost:11434/api/tags

# Run in verbose debug mode:
python atars_v2.py --input data.csv --city Test --no-llm --debug

# Verify audit log hash (reproducibility check):
python -c "import json,hashlib; j=json.load(open('atars_audit_xxx.json'));
print(j['hash_J'])"
```

17. Error Reference

Table 22: ATARS v2.0 error codes and messages

Error Code	Message	Cause	Recovery
E001	FileNotFoundError: input_file not found	CSV path does not exist	Verify --input path
E002	ValueError: No datetime column detected	No recognisable datetime column	Add datetime column or configure datetime_col
E003	ValueError: No valid pollutant columns found	All columns failed normalisation	Check column names against Table 3
E004	ValueError: config[z_threshold] must be >0	Invalid z_threshold in config	Set z_threshold ≥ 1.0
E005	ValueError: hw_alpha must be in (0,1)	α/β/γ outside valid range	Set smoothing parameters to values in (0,1)
W001	Quality flag: 0 valid records for variable X	All X records outside physical bounds	Check units; variable excluded from analysis
W002	Baseline: window too small for day d	Fewer than W//3 days available	Normal at year boundaries; result still valid
W003	IF skipped: fewer than 30 days with 5 variables	Insufficient multivariate data	Reduce if_min_days in config
W004	HW skipped: PM10 series too short	Fewer than 2×m days of PM10	Need ≥14 days of PM10 for seasonal init
W005	Ollama timeout after N attempts	LLM too slow for hardware	Increase llm_timeout or use --no-llm

18. Extending ATARS v2.0

18.1 Adding a New Variable

To add a new monitored variable (e.g., PM2.5):

1. **Add synonym aliases —** In normalise_columns(), add: "pm25", "PM2.5", "pm_25", "fine_pm" → "PM2.5"
2. **Add physical bounds —** In DEFAULT_CONFIG["physical_bounds"], add: "PM2.5": [0, 500]
3. **Add WHO threshold —** In load_who_thresholds(), add: "PM2.5": 15.0
4. **Rebuild J —** build_json_contract() picks up the new variable automatically
5. **Update charts —** chart_timeseries() already iterates over all detected variables — no change needed

18.2 Adding a New Chart

```
# 1. Create the function:
def chart_16_my_new_chart(daily_df, J, config, out_dir)  -> str:
    fig, ax = plt.subplots(figsize=(12, 5))
    # ... plotting code ...
    path = str(Path(out_dir) / "chart16_my_new_chart.png")
    fig.savefig(path, dpi=150, bbox_inches="tight")
    plt.close(fig)
    return path

# 2. Register in generate_all_charts():
    chart_paths["chart16"] = chart_16_my_new_chart(daily_df, J, config,
out_dir)

# 3. Add an insight in generate_chart_insights():
    insights["chart16"] = f"My chart shows ... {some_stat:.1f}"
```

18.3 Adding a New Report Section

```
# In build_report(), after section_18():
def section_19(doc, data, config):
    doc.add_heading("19.  My New Section", level=1)
    doc.add_paragraph("Content here...")

# Call it:
section_19(doc, my_data, config)
```

18.4 Replacing the LLM

To use a different LLM (e.g., OpenAI GPT-4, Anthropic Claude, a local GGUF model via llama.cpp), replace the POST request in try_llm_narrative() with your LLM's API call. The function signature and output format (narrative dict) must remain unchanged. The RGV operates on the output — it is LLM-agnostic and will work with any text-generating system.

19. Changelog and Known Issues

19.1 v2.0.0 — Current Release

- **ML Enhancement 1 —** Added Isolation Forest multivariate anomaly detection (Eq. 18). n_estimators=150, contamination=0.05, random_state=42.
- **ML Enhancement 2 —** Added Holt-Winters triple exponential smoothing 14-day PM10 forecast with 80% PI (Eqs. 19–23).
- **Novel contribution —** Runtime Grounding Verifier (RGV) with G_rate metric (Eqs. 15–16, 24–26). First algorithmic LLM grounding enforcement in AQ pipelines.
- **Report fix — section renumbering —** Sections renumbered 1–18 (previous: 1–13b–13c–18). All cross-references updated.
- **Report fix — all-caps openers —** 26 section openers rewritten from ALL-CAPS to proper research prose.
- **Report fix — ungrounded display —** Full sentence text displayed (500 chars with ellipsis), two-line format with section badge.
- **LLM fix — preamble cleaning —** _clean_llm_response() strips 14 preamble patterns. Threshold: 25% minimum length retention.
- **LLM fix — anti-preamble system prompt —** Rule 7 added: explicit list of 10+ forbidden openers. F_limitations prompt rewritten as 4 explicit sentence targets.
- **Security —** ML results excluded from J. Zero external transmission guaranteed. SHA-256 audit for both J and report.
- **Charts 14–15 —** New: IF vs z-score comparison chart and HW forecast chart added.

19.2 Known Issues

- **PACF equation wrapping —** In the generated Word report, Eq. 12 (PACF Yule-Walker) may wrap onto two lines on narrow page margins. Workaround: increase page margins to 1.25 inch.
- **HW seasonal period —** hw_seasonal_periods=7 assumes weekly seasonality. For hourly data, set to 24 for diurnal seasonality. Changing this invalidates the existing SHA-256 audit hash.
- **Ollama temperature=0 not always exact —** Some Ollama model versions ignore temperature=0 and produce slightly different outputs across runs. Use --no-llm for strict SHA-256 reproducibility of narrative.

19.3 Version History

Table 23: Full version history

Version	Date	Lines	Key Changes
2.0.0	2026-02-26	4,449	ML modules, RGV, report fixes, preamble cleaner (current)

Version	Date	Lines	Key Changes
1.5.0	2025-12	3,920	SHA-256 audit, Task Scheduler, security hardening
1.2.0	2025-11	3,400	15 charts, deterministic insights, chart 13 RGV dashboard
1.0.0	2025-10	2,800	Initial: 16 operators, 9-step pipeline, Word report

20. Glossary

Table 24: ATARS v2.0 terminology glossary

Term	Definition
A(D_v)	Aggregation operator — computes daily mean, std, and count for variable v (Eqs. 3–5)
ACF	Autocorrelation Function — measures temporal dependency at lag k (Eq. 11)
AQG	Air Quality Guideline — WHO 2021 annual reference concentration limits
ATARS	Automated Time-Series Analysis and Reporting System — this system
B(D,W)	Rolling Baseline operator — 30-day centred window mean and std (Eqs. 6–7)
BTEX	Benzene, Toluene, Ethylbenzene, Xylene — carcinogenic VOC group
CI	Confidence Interval — 95% interval around daily mean (Eq. 10)
contamination	Isolation Forest parameter — expected fraction of anomalies (default 0.05 = 5%)
δ_d	Delta deviation — percentage departure from rolling baseline (Eq. 9)
G(s)	Grounding function — set of J values matching claims in sentence s (Eq. 15)
G_rate	Grounding rate — fraction of numerical sentences grounded in J (Eq. 16)
IF	Isolation Forest — ensemble unsupervised multivariate anomaly detection (Eq. 18)
J	JSON contract — formal boundary between statistical layer and LLM layer
L	Lookup table — flattened {path: float} mapping built from J by RGV
OLS	Ordinary Least Squares regression (Eq. 14)
PACF	Partial Autocorrelation Function (Eq. 12)
PI	Prediction Interval — range for future observations (80% for HW forecast, Eq. 23)
Q(D)	Quality score — N_valid / N_total (Eq. 2). Range [0,1]
RGV	Runtime Grounding Verifier — novel contribution. Enforces G(s)⊆J algorithmically
σ_{W,d}	Baseline standard deviation — std of 30-day window around day d (Eq. 7)
ζ_d	Z-score — daily deviation from baseline in σ units (Eq. 8)
ζ_thresh	Z-score anomaly threshold — default 3.0. |ζ_d| > ζ_thresh = anomaly flag
α,β,γ	Holt-Winters smoothing parameters: level, trend, seasonal (Eqs. 19–21)

Term	Definition
ε	Tolerance — maximum allowed relative or absolute deviation for RGV grounding (Eq. 24)

— End of ATARS v2.0 System Documentation —

MIT License — Open Source. Copyright (c) 2026 Priyanshu.

ATARS v2.0.0 · atars_v2.py · 4,449 lines · 44 functions